Enterprise Integration Middleware

Event-driven middleware bridging a core trading platform with downstream operational services — Java 17, Spring Boot 3, Spring Integration, Spring Batch.

Java 17 · Spring Boot 3 · Spring Integration · Spring Batch 5 · Oracle DB · MapStruct · EhCache 3 · OpenTelemetry · GitLab CI · Ansible · Docker / Jib

1 System Overview

A production-grade **async middleware service** — integration layer between a core trading system and an operations platform. Inbound REST requests are persisted immediately to Oracle and processed asynchronously via Spring Batch, routed through dedicated Spring Integration message channels to domain handlers which delegate to downstream REST proxies.

Inbound Layer

- Spring MVC REST Controller
- Bean validation (@Valid)
- MapStruct DTO mapping
- Persist to Oracle (PENDING)
- Return HTTP 200 immediately

Integration Layer

- Spring Integration DirectChannel
- DynamicRecipientListRouter
- One MessageHandler per operation
- ServiceHelper (cross-cutting)
- Spring Retry on proxy calls

Outbound Layer

- Generated OpenAPI client stubs
- Hand-written proxy wrappers
- Apache HttpClient5 (SSL-aware)
- Reference data invoker (cached)
- EhCache 3 (cache-aside)

Decoupling intake from processing
Callers needed instant acknowledgement regardless of downstream latency. Solved with outbox pattern — persist first, process async via Spring Batch.

Operation-level routing without branching code
Routing dozens of operations cleanly — one DirectChannel + one MessageHandler per operation, wired via configuration, no switch statements.

Resilience against transient downstream failures
Spring Retry at the proxy layer with configurable attempt counts and back-off delays per service type handles transient 5xx and timeouts.

Zero data loss on JVM crash
A startup CommandLineRunner resets stale IN_PROGRESS records to FAILED — no request silently abandoned after an unexpected restart.

2 Async Request Lifecycle

```
Caller ──POST /api/operation──▸ REST Controller
                        │
                        ├─ Validate (@Valid)  ├─ Map entity (MapStruct)
                        ├─ Save to Oracle [PENDING]
                        └─ Return HTTP 200 immediately  ◂── caller is free

     Spring Batch Job  (asynchronously, in background)
       ├─ Read PENDING records from Oracle
       ├─ Group by priority  ──▸  ExecutorService parallel workers
       └─ @Transformer builds typed Message
                        │
         DynamicRecipientListRouter
                        │
            ┌───────────┼───────────┐
            ▼           ▼           ▼
     DirectChannel DirectChannel DirectChannel ... (one per operation)
            │
     OperationHandler
       ├─ EhCache lookup (reference data)
       ├─ Call downstream REST Proxy (Spring Retry)
       ├─ Update Oracle [SUCCESS / FAILED]
       └─ POST result to callback endpoint (optional)
```

Why not a message broker? In-process Spring Integration channels provide transactional consistency with the Oracle state store, zero broker operational overhead, and deterministic per-operation routing — all via properties, not code.

3 Spring Batch Jobs

Three jobs handle all async processing. Jobs are **not auto-started** — triggered via `JobLauncher` or `@Scheduled`. `spring.batch.job.enabled=false` prevents accidental startup execution.

Integration Job

Scheduled / Programmatic

Reads PENDING records from Oracle (up to 1500/cycle), groups by priority, submits each group to Spring Integration channels via a parallel ExecutorService thread pool.

Reference Data Sync Job

Scheduled

Synchronises reference data into a local Oracle cache table. Cached entries served from EhCache — sync job keeps them current and eliminates per-request round-trips.

Purge Job

Scheduled

Deletes completed and failed records older than the retention period (default 120 days). Bounds Oracle table growth without manual DBA intervention.

4 Database Design & Request State Machine

Oracle is the system of record. Every API call gets a row immediately on receipt — before any async processing. Schema managed via SQL migration scripts (`ddl-auto=none`).

Inbound API Call → **PENDING** Batch picks up → **IN_PROGRESS** proxy call → **SUCCESS** or **FAILED** purge job → **DELETED**

On JVM restart: any IN_PROGRESS record is reset to FAILED by a startup CommandLineRunner — no record silently abandoned.

Table	Purpose	Key Fields
Request Table	Top-level lifecycle row per inbound API call	requestId, operationName, status, priority, createdAt
Source Data Feed	Raw inbound payload stored immediately on receipt	requestId (FK), rawPayload, sourceSystem, receivedAt
API Request Details	Per-step result — request sent and response received	requestId (FK), apiName, requestBody, responseBody, status
Operation Registry	Master list of operation names and routing metadata	methodName, channelName, flowType, active
Reference Data Cache	Locally cached lookup data synced by Ref Data Sync Job	refType, refCode, cachedValue, lastSyncAt

5 Key Architecture Patterns

⚡ **Async Request-Reply (Outbox style)**

Caller gets instant HTTP 200. Oracle acts as the durable outbox — Spring Batch picks up and processes records async, decoupling intake from processing latency.

⇄ **Property-Driven Message Routing**

DynamicRecipientListRouter resolves the target DirectChannel at runtime from an externalized property — routing changed without code change or redeployment.

🔄 **Priority-Based Parallel Processing**

Spring Batch groups pending records by priority. ExecutorService processes each group concurrently, blocking before advancing — high-priority operations never starved.

🛡 **Startup Recovery (Fail-Safe)**

CommandLineRunner marks any IN_PROGRESS records as FAILED on JVM start — prevents silent data loss after unexpected restarts.

💾 **Cache-Aside (EhCache 3)**

Reference data cached in EhCache via Spring Cache abstraction — eliminates redundant round-trips during high-frequency batch cycles.

🔒 **Proxy Abstraction Layer**

Generated OpenAPI stubs wrapped in hand-written proxy classes adding SSL, retry, and error mapping — insulates business logic from generated code churn.

🌏 **Single-Artifact Multi-Environment**

One JAR across all environments. Spring Profiles + Jasypt-encrypted credentials externalize all config — no rebuilds, no env-specific artifacts.

6 Architecture Decisions

Decision	Choice	Rationale
Async mechanism	Spring Batch + Oracle	Transactional consistency with request-state DB; no broker to manage; retry via PENDING records
Channel routing	DirectChannel + RecipientListRouter	In-process, no serialization; one channel per operation; routing via properties
Client generation	OpenAPI codegen stub + proxy wrapper	Decouples business logic from generated code; centralizes SSL / retry / error handling
Container build	Maven Jib Plugin (no Dockerfile)	No Docker-in-Docker in CI; reproducible layered builds; faster pipeline
Credential security	Jasypt + env-var master password	Encrypted at rest in source; master password never in code or CI logs
Schema management	SQL scripts (ddl-auto=none)	Full DBA control; no accidental Hibernate DDL changes in production
Test isolation	H2 in-memory + schema.sql	Zero external dependencies in CI; Oracle-compatible DDL without Oracle license

7 CI/CD Pipeline

Build	→	Test	→	Image Push	→	Vuln Scan	→	Deploy	→	Verify
mvn install		H2 in-memory		Jib build		Container		Ansible		Health check
Auto		Auto		Auto		Auto		Manual		Auto

Ansible sequence: **stop** → **deploy** → **enable service** → **start** → **health check** → **proxy connectivity check**. PROD/PRE blocked in CI — requires DevOps release process.

8 Observability

Distributed Tracing
OpenTelemetry Java Agent auto-instruments HTTP, Spring Integration channels, and Spring Batch jobs. Trace context propagated end-to-end.

Metrics
Micrometer + Prometheus — JVM, HTTP, batch, and integration metrics at /actuator/prometheus.

Logging
Logback externalized outside JAR — runtime log-level changes without rebuild. Forwarded to Splunk from host or container stdout.

Health Check
Spring Boot Actuator at /actuator/health. Custom smoke-test endpoint verifies all downstream services are reachable.

API Documentation
Springdoc OpenAPI 2.6 — Swagger UI auto-generated from controller annotations at /swagger-ui.html.

Startup Recovery
CommandLineRunner marks IN_PROGRESS records as FAILED on restart — no request silently abandoned after a crash.

9 Technology Stack

Java 17 · Spring Boot 3.3.0 · Spring Integration · Spring Batch 5 · Spring Data JPA · Spring Retry · Spring Cache · Oracle DB (ojdbc17) · H2 (tests) · MapStruct 1.5.3 · Lombok · Apache HttpClient5 · EhCache 3 · Jasypt · Springdoc OpenAPI 2.6 · Micrometer + Prometheus · OpenTelemetry Java Agent · Splunk · Maven 3.8+ · Maven Jib Plugin · GitLab CI/CD · Ansible · Docker · JUnit 4 · Mockito